LAB 2

Encapsulation in C++

Setters, Getters, and Data Hiding

Learning Objectives

By the end of this lab, you should be able to:

- Explain what encapsulation is and why it matters in OOP.
- Use the private and public access specifiers correctly.
- Write setter functions to safely assign values to private data members.
- Write getter functions to safely retrieve values from private data members.
- Add validation logic inside setters to prevent invalid data.

1. Why Do We Need Encapsulation?

Imagine a class where every variable is **public**. Anyone using your class can read or change those variables freely. This sounds convenient, but it causes serious problems:

> ⚠ **Problems when data is public**
>
> Anyone can modify your data — even by accident.
> Invalid values can be assigned (e.g., a negative age, a salary of −5000).
> You have no control over how the data is used.
> If you later change how the data is stored, every program that uses your class breaks.

The Solution: Encapsulation

Encapsulation is the process of wrapping data (variables) and the functions that work on that data into one unit — a **class** — and restricting direct access to the data from outside.

> 📘 **Remember this formula**
>
> **Encapsulation = Data Hiding + Controlled Access**

Benefits of Encapsulation

- **Data security —** private data cannot be touched directly from outside the class.
- **Controlled access —** the class decides exactly how its data can be read or changed.
- **Easy maintenance —** internal changes don't break the code that uses the class.
- **Code flexibility —** you can add validation, logging, or limits without changing the interface.
- **Protection from accidental modification —** no more silent bugs from someone overwriting a variable.

2. Setters and Getters: The Tools

Encapsulation in C++ is built using three pieces working together:

- **private data members** — hide the actual variables inside the class.
- **public setter functions** — give a safe, controlled way to assign values.
- **public getter functions** — give a safe, controlled way to read values.

> **Quick reference**
> Setter → puts data IN to a private variable (with optional checks).
> Getter → takes data OUT of a private variable.
> Naming convention: setX(...) and getX(). Example: setId(101), getId().

3. Code 1 — Basic Encapsulation

Goal: build the simplest possible class that uses a setter and a getter to protect one private variable.

The Code

```
#include <iostream>
using namespace std;

class Student {
private:
    int id;   // private data member

public:
    // Setter function
    void setId(int i) {
        id = i;
    }

    // Getter function
    int getId() {
        return id;
    }
};

int main() {
    Student s1;
    s1.setId(101);                              // setting value
    cout << "Student ID: " << s1.getId();       // getting value
}
```

Code Breakdown

Code	What it does
`class Student {`	← Start defining a new class named Student.
`private:`	← Everything below this is hidden from the outside world.
`    int id;`	← The actual data — only the class itself can touch it directly.
`public:`	← Everything below this is accessible from outside the class.
`void setId(int i) {`	← Setter: takes a value i from the user.
`    id = i;`	← Stores it inside the private variable.
`int getId() {`	← Getter: a way to read the private value safely.
`    return id;`	← Hands the stored value back to the caller.
`Student s1;`	← Create an object of class Student.
`s1.setId(101);`	← Use the setter to put 101 inside s1's id.
`s1.getId();`	← Use the getter to read s1's id back out.

Expected Output

▶ Output

```
Student ID: 101
```

> 📘 **Try this yourself**
>
> What happens if you write s1.id = 101; directly in main()? Try it — you'll get a compiler error: 'id is private within this context'. That error is encapsulation doing its job.

4. Code 2 — Multiple Variables

Goal: extend the idea to a class with more than one private variable, and use a single function to set them all at once.

The Code

```
#include <iostream>
using namespace std;
```

```
class Employee {
private:
    int empId;
    double salary;

public:
    void setData(int id, double sal) {
        empId  = id;
        salary = sal;
    }

    void display() {
        cout << "Employee ID: " << empId  << endl;
        cout << "Salary: "      << salary << endl;
    }
};

int main() {
    Employee e1;
    e1.setData(1, 50000);
    e1.display();
}
```

Code Breakdown

Code	What it does
`int empId;`	← First private variable — the employee's ID.
`double salary;`	← Second private variable — uses double to allow decimals.
`void setData(int id, double sal) {`	← One setter that takes BOTH values at once. Saves writing two separate setters.
`    empId  = id;`	← Copy the parameter into the private variable.
`    salary = sal;`	← Same idea for salary.
`void display() {`	← A helper member function that prints both values neatly.
`Employee e1;`	← Create one Employee object.
`e1.setData(1, 50000);`	← Pass both values in one call: id = 1, salary = 50000.
`e1.display();`	← Ask the object to print itself.

Expected Output

▶ Output

```
Employee ID: 1
Salary: 50000
```

> **Two styles of setters**
>
> Style A — one setter per variable: setEmpId(...), setSalary(...). Best when each value is set at a different time.
>
> Style B — one combined setter: setData(...). Best when values are always provided together.
>
> Both are valid. Pick whichever fits the problem.

5. Code 3 — Encapsulation with Validation

This is where encapsulation really pays off. A setter is not just an assignment — it can **check the value first** and refuse bad input.

The Code

```
#include <iostream>
using namespace std;

class BankAccount {
private:
    double balance;

public:
    void setBalance(double b) {
        if (b >= 0)
            balance = b;
        else
            cout << "Invalid balance!" << endl;
    }

    double getBalance() {
        return balance;
    }
};

int main() {
    BankAccount acc;
    acc.setBalance(1000);
    cout << "Balance: " << acc.getBalance();
}
```

Code Breakdown

Code	What it does
`double balance;`	← Private — no one outside can change it directly.
`void setBalance(double b) {`	← Setter receives the proposed new balance.
`    if (b >= 0)`	← VALIDATION: only allow values that are zero or positive.
`        balance = b;`	← If the check passes, store the value.
`    else`	← Otherwise...
`        cout << "Invalid...";`	← ...refuse the value and warn the user.
`double getBalance() {`	← Getter — simply returns whatever is currently stored.
`acc.setBalance(1000);`	← 1000 is valid (≥ 0), so it gets stored.

Expected Output

▶ Output

```
Balance: 1000
```

⚠ **What if you tried setBalance(-500)?**

Output would be: Invalid balance!

And the previous valid value of balance would stay unchanged. That is the whole point — bad data never reaches the variable.

📘 **Key idea**

Without encapsulation: acc.balance = -500; silently corrupts the object.

With encapsulation: the setter is the gatekeeper. Bad data is rejected before it can do damage.

6. Code 4 — A Harder Problem (Rectangle)

Now we combine everything: multiple private variables, individual setters, individual getters, validation, AND a member function that uses the private data to compute something useful.

Requirements

Class: Rectangle

Private data members:

- length

- width

Public member functions:

- Setters to assign length and width.
- Getters to retrieve length and width.
- A function to calculate and return the area.

> ⚠ **Validation rules**
>
> Length and width must both be positive numbers (> 0).
> If the user enters an invalid value, display an error message and do not store the value.

The Code

```
#include <iostream>
using namespace std;

class Rectangle {
private:
    double length;
    double width;

public:
    // Setter for length
    void setLength(double l) {
        if (l > 0)
            length = l;
        else
            cout << "Invalid length! Must be positive." << endl;
    }

    // Setter for width
    void setWidth(double w) {
        if (w > 0)
            width = w;
        else
            cout << "Invalid width! Must be positive." << endl;
    }

    // Getter functions
    double getLength() {
        return length;
    }

    double getWidth() {
        return width;
    }

    // Function to calculate area
    double calculateArea() {
```

```
        return length * width;
    }
};

int main() {
    Rectangle r;
    r.setLength(5);
    r.setWidth(4);

    cout << "Length: " << r.getLength()    << endl;
    cout << "Width: "  << r.getWidth()     << endl;
    cout << "Area: "   << r.calculateArea() << endl;

    return 0;
}
```

Code Breakdown

Code	What it does
`double length;`	← Private — width and length are hidden.
`double width;`	← Same — only the class can touch it.
`void setLength(double l) {`	← Each variable gets its OWN setter this time.
`if (l > 0)`	← Strictly positive — zero is not a valid length.
`length = l;`	← Accept the value if the check passes.
`else cout << "Invalid...";`	← Reject with a clear message if it fails.
`void setWidth(double w) {`	← Same pattern repeated for width.
`double getLength() {...}`	← Plain getters — just hand the value back.
`double calculateArea() {`	← A member function that USES the private data.
`return length * width;`	← Because it's inside the class, it can read length & width directly.
`Rectangle r;`	← Make a Rectangle object.
`r.setLength(5); r.setWidth(4);`	← Both 5 and 4 pass validation, so they're stored.
`r.calculateArea();`	← Returns 5 × 4 = 20.

Expected Output

▶ Output

```
Length: 5
```

```
Width: 4
Area: 20
```

> **Why is calculateArea() a member function?**
> It could be a free function that takes a Rectangle, but making it a member keeps related behavior bundled with the data. That bundling IS encapsulation.

7. Summary

Three things to take away from this lab:

1. **Hide the data.** Use private for any variable that should not be modified directly from outside the class.
2. **Expose controlled access.** Use public setter and getter functions to read and write that data.
3. **Validate inside the setter.** A setter is the perfect place to enforce rules (positive only, range 0–100, etc.).

8. Practice Tasks

Task 1 — Person

Create a class Person with the following:

- Private data members: name (string), age (int).
- A setter and a getter for each variable.
- A function display() that prints the person's name and age in a clean format.

> **Hints for Task 1**
> #include <string> and use the string type for name.
> Pass strings to the setter by value (or by const reference if you've covered it).
> Test with at least two different Person objects in main().

Task 2 — Student with Grade

Create a class Student with the following:

Private data members:

- name
- marks

Public member functions:

- Setter for name.
- Setter for marks (with validation).
- Getters for name and marks.

- A function displayGrade() that prints a letter grade based on marks.

⚠ **Validation rule**

Marks must be between 0 and 100 (inclusive). If invalid, show an error and do not store the value.

Suggested grading scale

90–100 → A 80–89 → B 70–79 → C 60–69 → D below 60 → F

Use if / else if inside displayGrade(). Test with marks like 95, 72, 55, and an invalid value like 120.

Prepared by

Alifa Shanzidah Manarat

Lecturer

Dept. of CSE, BAUST